

Projet TPRE101 — Service web de livrables

“Projet de groupe : Développement d'une solution Web”

Projet du 23/03/2025



l'école d'ingénierie
informatique



l'école [tech]
de l'expertise digitale

Participants :

Kaélian BAUDELET
Annalia PRIEUR
Florian GUILBERT
Thibault ROYER

**Conception et réalisation d'une
plateforme de livrable — Première
Année, 2025**

EPSI — Ecole de l'ingénierie
informatique, Arras

Projet encadré par :

Grégory BOUDRINGHIN

CONSIGNE DU PROJET

Projet transversal Web SN1

TPRE101 – Conception et Développement d'une solution web En utilisant : HTML, CSS, PHP

Réaliser un service web de livrables qui n'accepte que l'envoi de fichiers ZIP et de fichiers powerpoint Le projet devra être Responsive pour s'adapter à n'importe quel type d'appareil et d'écran

Le projet doit contenir :

Une page d'administration (admin.php) sous la forme d'une formulaire Web qui permet de CRUD sur le serveur des dossiers pour les livrables.

Pour le CREATE: Il faut pouvoir créer un dossier en précisant la section concernée et mettre une date limite pour la livraison,

Pour le READ : La page d'administration doit également afficher sous forme d'arborescence les fichiers et dossiers de livrables présents sur le serveur et permettre de les télécharger localement,

Pour le UPDATE : Il faut pouvoir changer la date de livraison,

Pour le DELETE : Les dossiers doivent pouvoir être détruit MAIS par sécurité deux messages d'avertissement devront s'afficher avant : «Etes-vous sûr(e) de vouloir détruire ce dossier ? » et « La destruction de ce dossier est définitive ? Confirmer ? »

Côté utilisateur (index.php) Une page de formulaire HTML5 permettra d'afficher l'arborescence des dossiers et uniquement des dossiers pour y choisir où envoyer le livrable. Un bouton d'envoi des livrables dans un dossier choisi apparaîtra dans l'arborescence affichée juste à droite de la ligne concernée.

L'ensemble devra être charté aux couleurs et logos EPSI - WIS

MODALITÉS DU PROJET

- Durée de rendu: **23/03/25**
- Ressources :
 - Ressources documentaires numérique en ligne
 - Plateforme 360Learning
 - Documentation officielle PHP
 - Bibliothèque ENI
- Objectifs et livrables :
 - **Site web fonctionnel** : Le site web fonctionnelle qui sera utilisés pour la présentation oral
 - **Maquettes de l'application** : Les maquettes et wireframe de l'application
 - **MCD/MLD de la base de donnée** : Les schémas MLD et MCD relationnelles de la base de données
 - **Export de la base données** : Exportation de la base de données sous forme de fichier .sql
 - **Dossier détaillé du projet** : Le dossier complet du projet détaillé.
 - **Présentation finale avec support** : La présentation accompagnée d'un support visuel sous forme de diapositives et devra être présentée oralement
 - **Code de l'application** : Code complet du projet propre et organisé.

Table des matières

1. Présentation de la plateforme	08
1.1. Contexte	08
1.2. Présentation du site	09
2. Spécifications techniques	17
2.1 Brainstorming et idées du projet	17
2.2 Base de donnée	17
2.3 Spécifications techniques du code	20
2.3.1 Outils utilisé	20
2.3.2 Explications techniques de l'IHM	22
2.3.2.1 Explications globales	22
2.3.2.2 Explications spécifiques au routeur	24
2.3.2.3 Explications spécifiques au modèle	26
2.3.2.4 Explications spécifiques aux entity	26
2.3.2.5 Explications spécifiques des contrôleurs	27
2.3.2.6 Explications spécifiques à la vue	27
3. Les points critiques	29
3.1 Points positifs et adéquation avec le sujet	29
3.2 Points négatifs	29
Ressources documentaires et références utilisés.....	31
Annexes.....	32

PROJET TPRE101 — Service web de livrables



Plateforme web de livrables



*FileSphere ne fait l'objet d'aucun dépôt et ne constitue pas une marque existante. Son seul objectif ici est d'être utilisé comme nom pour une plateforme fictive créée à des fins de démonstration.

Table des Annexes

Le dossier de projet suivant contient plusieurs annexes regroupant plusieurs images, illustrations et schémas.

Dictionnaire de données.....	33
Dépendances fonctionnelles.....	35
Schéma MCD.....	36
Schéma MLD.....	37
SQL.....	38
Wireframes.....	40

Avant propos

Dès lors que ce dossier de projet aura été publié sur la plateforme 360Learning, une version publique de la plateforme de livrables *FileSphere* sera disponible pour vous permettre de la tester en détail.

Vous pourrez accéder à la plateforme via le lien suivant : <https://filesphere.kaelian.dev>.

Pour vous connecter, utilisez les identifiants ci-dessous selon le rôle que vous souhaitez tester :

- Compte **Admin** :

Email : admin@exemple.com / Mot de passe : password123

- Compte **Intervenant** :

Email : teatcher@exemple.com / Mot de passe : password123

- Compte **Élève** :

Email : student@exemple.com / Mot de passe : password123

Une fois connecté à la plateforme de démonstration, vous aurez la possibilité d'explorer toutes ses fonctionnalités et de tester librement les différentes options disponibles. Vous pourrez ainsi vous familiariser avec l'interface, effectuer des actions et vous faire une idée complète du fonctionnement de la plateforme.

Cependant, veuillez noter que toutes les modifications apportées à la version de démonstration seront automatiquement réinitialisées à la fin de chaque journée. Cela signifie que toute donnée ajoutée, toute configuration modifiée ou toute action effectuée sera supprimée et que la plateforme de démonstration reviendra à son état initial.

1. Présentation de la plateforme

1.1 Contexte

Dans le cadre du module **TPRE101 – Conception et Développement d'une solution web**, il est demandé de réaliser une application web permettant la gestion et le dépôt de livrables. Ce projet doit être conçu en utilisant **HTML, CSS et PHP**.

L'objectif principal est de proposer un service web dédié à la gestion des livrables, qui accepte exclusivement les fichiers ZIP et PowerPoint. Il devra être responsive, s'adaptant à tout type d'écran et d'appareil.

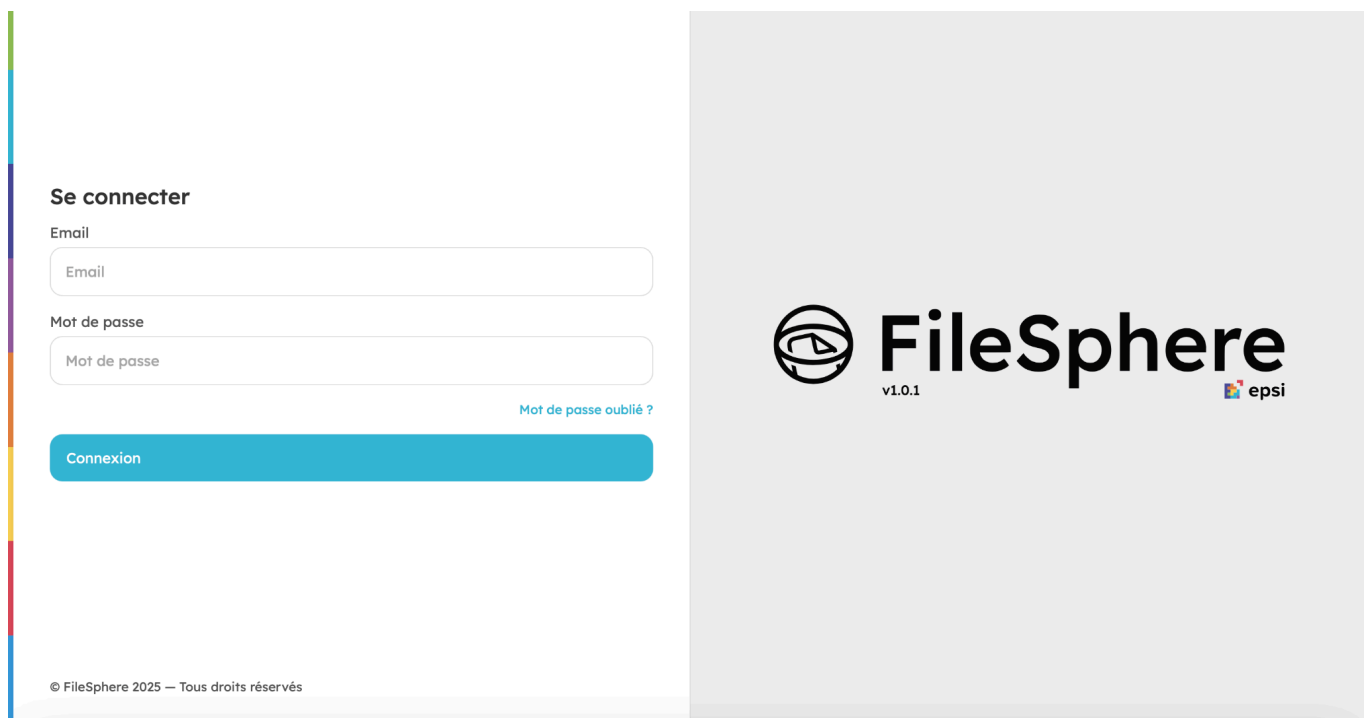
Le projet comprend deux interfaces principales :

1. Une interface d'administration permettant la gestion des dossiers de livrables. L'administrateur pourra :
 - Créer un dossier en spécifiant la section concernée et en définissant une date limite de livraison.
 - Consulter l'arborescence des dossiers et fichiers présents sur le serveur, avec possibilité de les télécharger.
 - Modifier la date limite de livraison d'un dossier.
 - Supprimer un dossier avec un double message de confirmation pour éviter toute suppression accidentelle.
2. Une interface utilisateur (index.php) où l'utilisateur pourra :
 - Visualiser l'arborescence des dossiers disponibles pour y déposer un livrable.
 - Sélectionner un dossier et envoyer un fichier ZIP ou PowerPoint directement via un formulaire.
 - L'ensemble du projet devra être conforme à la charte graphique de EPSI/WIS, intégrant leurs couleurs et logos.

1.2 Présentation du site

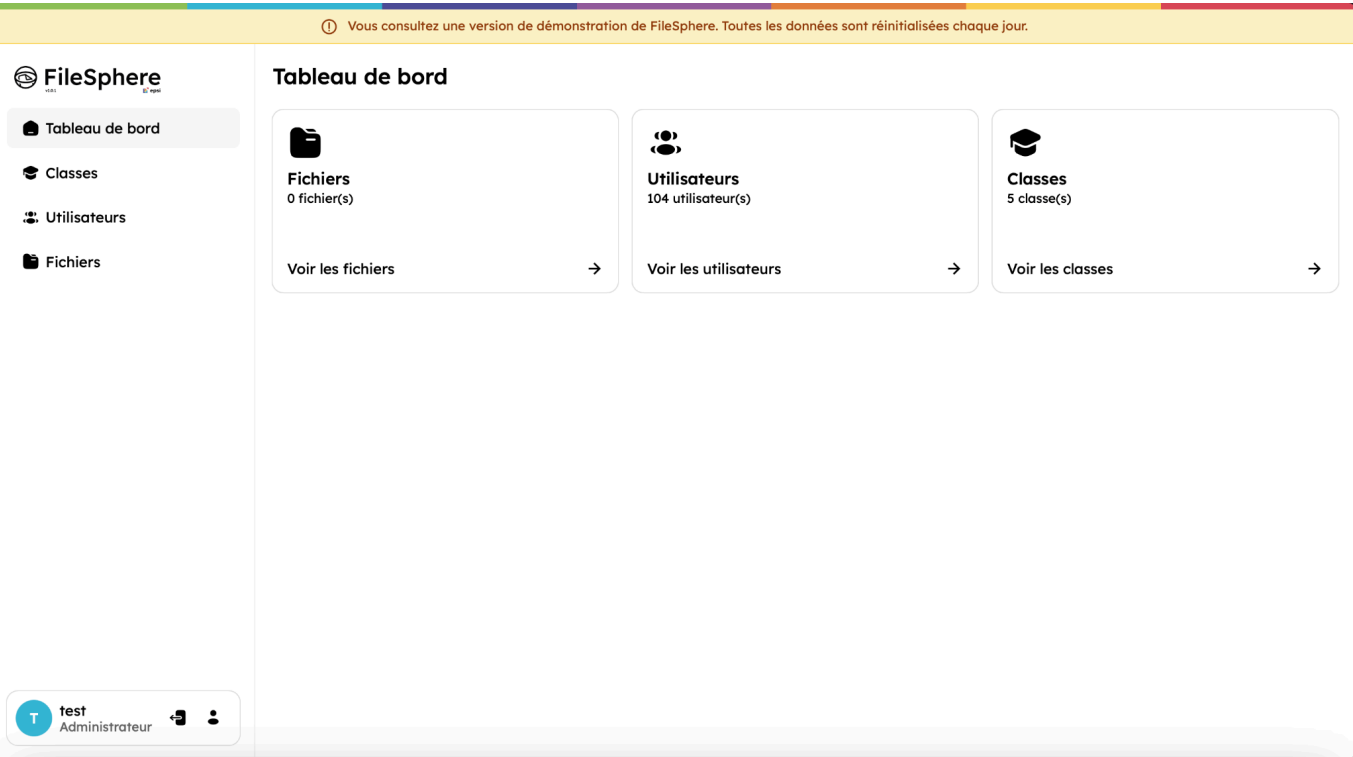
La première page sur laquelle l'utilisateur arrive est la page de connexion. En effet, l'accès au site nécessite une connexion préalable.

Cependant, dans le cadre de la phase de test, nous avons également intégré une fonctionnalité permettant d'activer ou désactiver l'inscription des nouveaux utilisateurs via le fichier de variable d'environnement `.env`. Il est donc nécessaire d'avoir accès au code pour modifier la variable `temp_register`. Si cette variable est définie sur `true`, la création de nouveaux comptes sera possible depuis l'interface du site via `/register`. Si elle est définie sur `false`, l'inscription sera désactivée et renverra une erreur 404. Ce système est temporaire et conçu pour permettre la création de comptes uniquement pendant la phase de test. En production, les comptes doivent être créés manuellement par les administrateurs. Une fois un compte créé par un administrateur, l'utilisateur reçoit un email avec un lien pour réinitialiser son mot de passe et finaliser son accès.

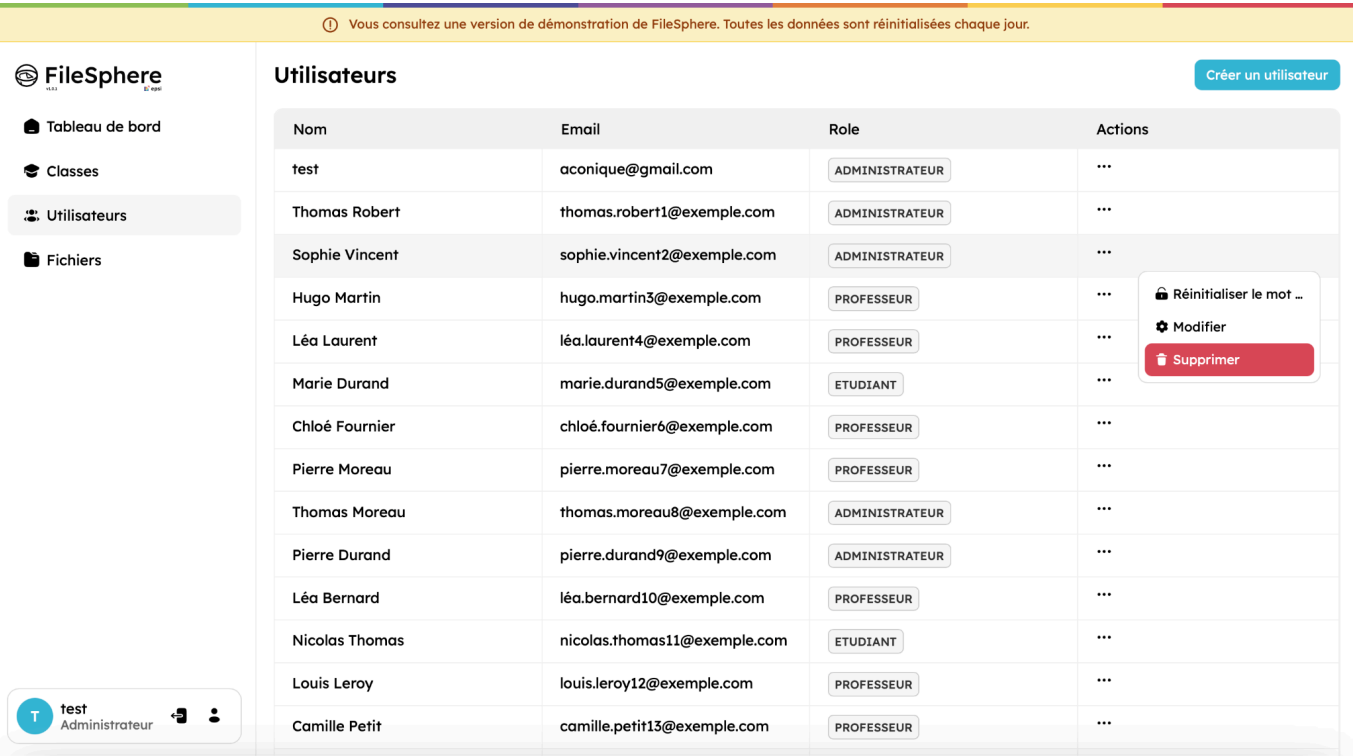


Ci-dessus, vous pouvez voir la page principale, la page de connexion. Elle présente le logo de l'EPSI ainsi que les couleurs emblématiques de l'établissement, avec le bleu clair de l'EPSI comme teinte principale. Nous avons choisi d'adopter un design simple et minimaliste, en privilégiant une majorité de blanc et de gris, afin de conserver un aspect sobre et élégant.

Une fois connecté nous arrivons sur le tableau de bord !



Ici l'on a tout d'abord la vue administrateur ! Ce dernier a accès à tous, que ce soit les fichiers, les classes, les devoirs et même les utilisateurs. Ils ont, bien évidemment, droit à supprimer, modifier et créer les classes, les utilisateurs, les fichiers et les devoirs.



Par exemple ici l'on a le point de vue d'un administrateur sur les utilisateurs, il peut CRUD (Create, Read, Update, Delete). Si l'on y regarde de plus près, Il y a aussi l'option de réinitialiser le mot de passe, cela permettant d'envoyer par mail le lien pour modifier le mot de passe de l'utilisateur. L'administrateur peut aussi modifier l'utilisateur, que cela soit son nom, prénom ou son mail.

Enfin, le côté important est la création de compte par l'administrateur

The screenshot shows a modal window titled "Créer un utilisateur" with a close button (X) in the top right corner. The form contains three input fields: "Nom" with a placeholder "Nom de l'utilisateur", "Email" with a placeholder "Email de l'utilisateur", and "Role du compte" with a dropdown menu currently showing "Etudiant". At the bottom of the modal is a large blue button labeled "Créer". In the background, a table of users is visible with columns: Nom, Email, Role, and Actions. The first row shows "test", "aconique@gmail.com", "ADMINISTRATEUR", and a three-dot menu icon.

Vous pouvez remarquer que la création de compte n'a pas de mot de passe, en effet dès qu'un utilisateur est créé il reçoit un mot de passe temporaire qu'il devra changer lors de sa première connexion.

Et voilà tout pour la gestion des utilisateurs ! Passons maintenant aux classes.

The screenshot shows the FileSphere web application interface. At the top, a yellow banner reads: "Vous consultez une version de démonstration de FileSphere. Toutes les données sont réinitialisées chaque jour." On the left is a sidebar with the FileSphere logo and navigation links: "Tableau de bord", "Classes" (selected), "Utilisateurs", and "Fichiers". The main area is titled "Classes" and features a "Créer une classe" button in the top right. Below this, there are four class cards: "MAT... Mathématiques" by Hugo Martin, "PHYS... Physique" by Hugo Martin, "CHI... Chimie" by Hugo Martin, and "BIOL... Biologie" by Hugo Martin. A context menu is open over the "Physique" card, showing options: "Gestion des étudia...", "Modifier", and "Supprimer".

Les classes suivent le même système, bien que l'affichage soit différent. Il est ici possible de modifier une classe ainsi que ses étudiants.

Concernant l'arborescence, elle n'est pas présentée de manière physique, mais elle fonctionne selon le même principe. Il suffit de cliquer sur une classe pour afficher ses sections, puis les devoirs associés. Ce fonctionnement repose sur une arborescence simple, inspirée du modèle de la file d'Ariane.



Tableau de bord

Classes

Utilisateurs

Fichiers

Classes > Mathématiques > Sections

Sections de Mathématiques

Calcul différentiel

Algèbre

Statistiques

Trigonon

Géométrie

Modifier

Supprimer

Créer une section

Et pareil pour les sections, où l'on peut cliquer dessus pour consulter les devoirs ou cliquer sur les 3 petits points afin de supprimer ou de modifier les sections. Il est également important de noter que les enseignants ont la même vision que l'administrateur pour pouvoir créer ou supprimer des sections.



Tableau de bord

Classes

Utilisateurs

Fichiers

Classes > Mathématiques > Sections > Algèbre > Devoirs

Devoir de la section Algèbre de la classe Mathématiques

Analyse de documents

Étude de cas

Expérience laboratoire

Créer un devoir

Lorsque l'on consulte les devoirs, ceux-ci possèdent la même fonctionnalité de modification ou de suppression. Il est également important de noter que les enseignants ont la même vision que l'administrateur pour pouvoir créer des devoirs et assigner les instructions correspondantes.

Maintenant que nous avons exploré les classes, revenons aux fichiers. L'administrateur a une vue d'ensemble des fichiers téléchargés sur la plateforme et peut ainsi ajouter ou supprimer des fichiers selon ses besoins.



Tableau de bord

Classes

Utilisateurs

Fichiers

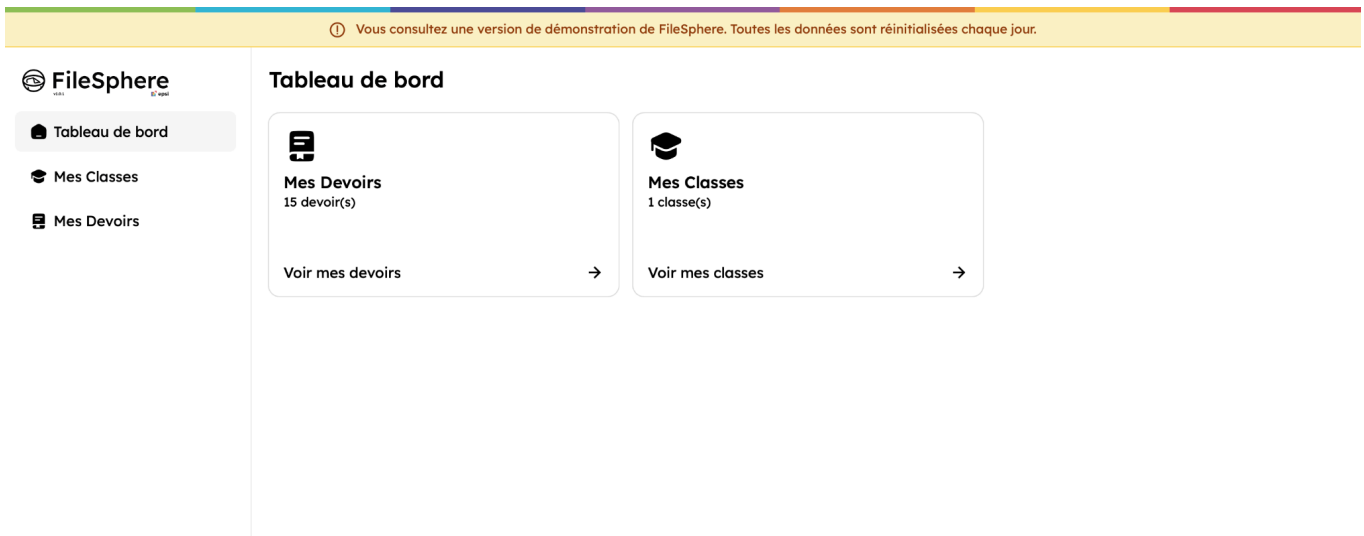
Fichiers

Fichier(s) téléchargé(s) avec succès.

Fichier	Taille	Propriétaire	Date de mise en ligne	Actions
logo.png	19.23 KB	test	23/03/2025 22:53	...

Téléverser un fichier

Penchons-nous maintenant sur la vue étudiante. Elle est légèrement différente, mais présente sensiblement les mêmes pages que vous pouvez remarquer. Cependant, elle n'aura pas de permissions d'édition ni de suppression et ne verra pas ces options.



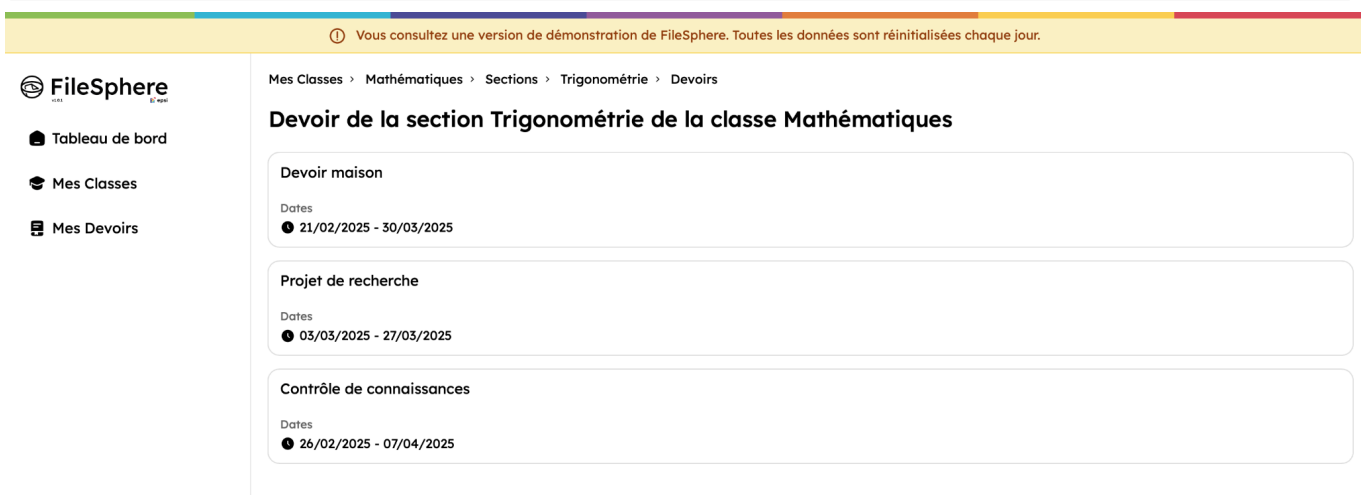
L'élève peut alors consulter rapidement ses devoirs ou ses classes depuis cette vue. Commençons par examiner les classes de l'élève : il peut ainsi accéder à la liste des classes auxquelles il est inscrit.



Ensuite, l'élève peut accéder directement aux sections d'une classe en cliquant dessus.



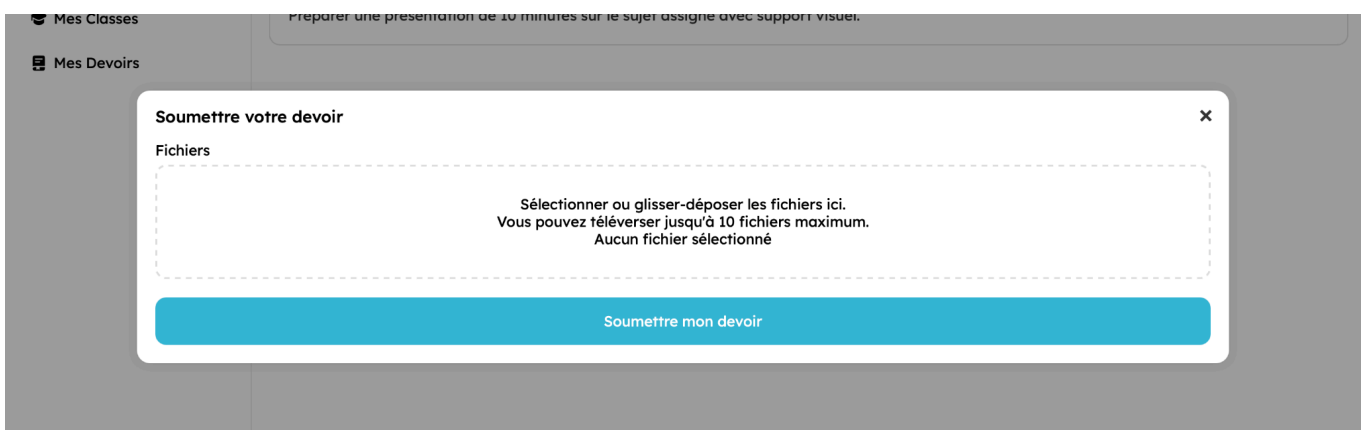
Et de la même manière, en cliquant sur une section, il accède aux devoirs de cette même section.



L'élève peut alors consulter ces devoirs à faire et cliquer dessus pour avoir les détails du devoirs avec les éventuelles instructions données par le professeurs.



L'élève peut alors soumettre son devoir en cliquant sur "Soumettre mon devoir", puis en sélectionnant un ou plusieurs fichiers à remettre.



Lorsque l'élève a soumis son travail, il peut également annuler sa soumission et supprimer le ou les fichiers s'il a choisi un mauvais fichier par erreur (à condition que le devoir n'est pas terminé), en utilisant le bouton "Annuler la remise" comme indiqué.

 FileSphere

Tableau de bord

Mes Classes

Mes Devoirs

Mes Classes > Mathématiques > Sections > Trigonométrie > Devoirs > Devoir maison

Devoir maison

Annuler la remise

Préparer une présentation de 10 minutes sur le sujet assigné avec support visuel.

Votre soumission :

devoir.png

Télécharger

test Etudiant

Nous avons passé en revue les pages liées aux classes de l'élève. Nous pouvons maintenant revenir à la page “Mes devoirs”, qui affiche en une seule vue l'ensemble des devoirs de l'élève, répartis sur plusieurs de ses classes, lui permettant ainsi d'avoir une vision globale de ses devoirs.

 FileSphere

Tableau de bord

Mes Classes

Mes Devoirs

Mes Devoirs

Analyse de documents

Dates

10/03/2025 - 02/04/2025

Étude de cas

Dates

12/03/2025 - 14/04/2025

Expérience laboratoire

Dates

25/02/2025 - 05/04/2025

Dissertation

Dates

26/02/2025 - 26/03/2025

Devoir maison

Dates

01/03/2025 - 27/03/2025

Présentation orale

Dates

04/03/2025 - 17/04/2025

test Etudiant

Enfin, une dernière page, qui n'a pas encore été abordée mais qui est présente sur tous les comptes, est la page “Profil”. Elle permet à l'utilisateur de modifier son mot de passe et de consulter ses informations de compte.

 FileSphere

Tableau de bord

Mes Classes

Mes Devoirs

test

Etudiant

<

2. Spécifications techniques

2.1 Brainstorming et idées du projet

Notre projet a d'abord commencé par une phase de brainstorming durant laquelle nous avons réfléchi en détail au rôle de l'application et à la bonne compréhension des consignes demandées avant de nous lancer dans le code. Nous avons ainsi sélectionné les fonctionnalités à développer, en mettant l'accent sur celles qui seraient les plus importantes. Les fonctionnalités importantes retenues pour ce projet sont les suivantes :

Un **crud (create, read, update, delete)** pour chaque table,

Une **fonctionnalité d'upload** de fichier sécurisé

Un **site sécurisé** grâce à différents **rôles** à permissions différentes

Une **sécurisation** de chaque entrée pour éviter **les attaques les plus commune**

Un **système de connexion et de vérification** de compte par **email**

2.2 Base de donnée

Pour commencer le projet, nous avons dû concevoir la base de données. Cette dernière nous a permis de définir tout ce dont nous aurions besoin pour la suite et de poser à plat ce que nous devons réaliser.

C'est ainsi que nous avons commencé par créer le dictionnaire de données, **fourni en annexe**. Dans ce dernier, nous avons déjà déterminé le nombre de tables nécessaires pour mettre en œuvre les fonctionnalités que nous avons définies en amont.

Nous avons créé les tables suivantes :

User

La table **User** contient les informations des utilisateurs. Elle possède un identifiant unique (**id**) généré avec un UUID, garantissant que chaque utilisateur est identifiable de manière distincte. Le champ **role** définit le rôle de l'utilisateur, qui peut être admin, teacher ou student, permettant de gérer les droits d'accès et les permissions. **name** et **email** stockent respectivement le nom complet de l'utilisateur et son email, qui doit être unique pour éviter les doublons. Le champ **password** est utilisé pour stocker le mot de passe de l'utilisateur de manière sécurisée. Le booléen **is_active** indique si le compte est actif ou non. Par défaut, il est mis à 0 (inactif) jusqu'à ce que l'utilisateur change son mot de passe initial. **reset_token** est utilisé lors des demandes de réinitialisation de mot de passe, et **reset_token_expires_at** définit la date d'expiration du token pour cette réinitialisation. Les champs **created_at** et **updated_at** permettent de suivre respectivement la date de création et la date de mise à jour du compte utilisateur.

Class

La **table Class** gère les informations liées aux classes dans le système. Chaque classe a un identifiant unique **id** (UUID), un **teacher_id** faisant référence à l'enseignant responsable, et un **file_id** optionnel pour associer un fichier à la classe, comme une présentation ou un document spécifique. La **color** permet de définir une couleur distincte pour chaque classe, tandis que **name** désigne le nom de la classe (ex. : "Mathématiques", "Informatique"). Les champs **created_at** et **updated_at** assurent le suivi temporel de la création et de la mise à jour de la classe. Les relations sont gérées par des clés étrangères avec les tables **User** pour l'enseignant et **File** pour les fichiers. Cette table est cruciale pour l'organisation des classes, l'attribution des enseignants, ainsi que la gestion des fichiers associés à chaque classe.

File

La table **File** stocke les fichiers téléchargés par les utilisateurs. Chaque fichier possède un **id** unique, un **token** pour un identifiant unique de téléchargement, le **name** du fichier, **extension** (type de fichier, comme pdf), et **size** (taille du fichier). **user_id** fait référence à l'utilisateur qui a téléchargé le fichier, créant une relation avec la table **User**. Les champs **created_at** et **updated_at** suivent respectivement la création et la mise à jour des fichiers.

Section

La table **Section** gère les sections dans une classe (elle peut être assimilée à des chapitres/matières/thèmes). Elle possède un **id** unique, un **name** qui est le nom de la section (par exemple, "Mathématiques", "Informatique"), et les champs **created_at** et **updated_at** pour suivre respectivement la création et la mise à jour de la section.

Assignment

La table **Assignment** contient les devoirs attribués aux sections. Chaque devoir possède un **id** unique, un **name** (titre du devoir), une **description** facultative, et des dates de **start_period** et **end_period** pour définir la période du devoir. **max_files** spécifie le nombre maximal de fichiers autorisés pour la soumission, et **allow_late_submission** détermine si les soumissions tardives sont autorisées. Les champs **created_at** et **updated_at** enregistrent respectivement la date de création et de mise à jour du devoir.

Relations et contraintes

User :

Dépend de :

Aucune table directement.

Relation avec :

- Un utilisateur peut être lié à plusieurs fichiers (car il peut être propriétaire de plusieurs fichier)
- Un utilisateur (enseignant) peut être responsable de plusieurs classes.
- Un utilisateur (élève) peut être inscrit dans plusieurs classes.

File :

Dépend de :

User (chaque fichier appartient à un utilisateur).

Relation avec :

- Un fichier peut être associé à une classe.
- Un fichier peut être lié à un devoir.

Class :

Dépend de :

- User : Un enseignant administre forcément une classe
- File : Optionnellement, une classe peut avoir un fichier associé (une icône)

Relation avec :

- Un utilisateur (élève) peut être lié à plusieurs classes.
- Une classe peut être associée à plusieurs sections.
- Une classe peut avoir plusieurs élèves.

Section :

Dépend de :

Aucune table directement.

Relation avec :

- Une section peut être liée à plusieurs classes.
- Une section peut avoir plusieurs devoirs.

Assignment :

Dépend de :

Aucune table directement.

Relation avec :

- Un devoir peut être associé à plusieurs sections.
- Un devoir peut avoir plusieurs fichiers associés.

2.3 Spécifications techniques du code

2.3.1 Outils utilisé

Pour mettre en application cela nous utilisons les technologies apprises en cours:



Cependant nous ne les avons pas utilisés **uniquement** parce que ces dernières étaient les plus simples d'approche pour le groupe et demandé dans la demande, elles sont aussi très pertinentes pour répondre au projet.

Twig sert à créer des pages plus dynamiques que du **html** simple en fonctionnant en système de bloc. Nous permettant ainsi de garder une unicité pour toutes les pages sans copier coller une énormité de code tel que l'on devrait faire pour des pages **html** simples. Son fonctionnement peut être vu tout au long du code, il est la base de la conception des pages et de leur fonctionnement. Il nous impose notamment un code en **MVC (Model View Controller)**, où nous devons, pour interagir avec la **base de donnée** / Permettre la gestion de variable partout sur le site et le contrôle de ces dernières au travers de différentes pages : le **model** qui permet ici de récupérer toutes les données à afficher ; la **vue**, qui sont les pages **html.twig**, qui ont l'interface graphique ; et enfin le **contrôleur** permet de comprendre les interactions de l'utilisateur avec la page et de relier le model à la vue graphique.

Nous utilisons aussi, pour gérer toutes les pages, un système que **M. Legales** nous a appris qui se rapproche des technologies utilisées pour **Symphony** notamment le **routing des pages** qui nous permet de gérer quels utilisateurs peuvent accéder à quelles pages. Il nous permet aussi d'interagir, avec le langage **SQL**, avec la base de données présentée plus haut.

Pour travailler en équipe, bien que nous ayons fait face à plusieurs problèmes de dernières minutes, nous avons travaillé avec le logiciel **Git** afin de pouvoir mettre en commun notre code et gagner du temps comme ce dernier nous permet de voir directement les changements et de sélectionner lesquels l'on garde grâce au git merge.

De plus, Kaelian ayant de plus ample connaissances, nous avons, grâce à lui, intégré d'autres technologies tel que:



afin de pouvoir développer en environnement local et faire tout les tests sans être dépendant de l'infrastructure donnée par l'EPSI (le nuage pédagogique)

Nous avons fait cela notamment pour pouvoir tester plus rapidement notre code, permettre de le faire même hors ligne, sauvegarder le tout localement mais aussi pour pouvoir apprendre de nouvelles technologies.

Cependant le tout venait tout de même avec des difficultés: Une nécessité d'un ordinateur avec de la place de stockage et qui tient la route pour faire tourner une "machine virtuelle" sous linux qui fait donc tourner le site en fond; un certain temps d'initiation à la technologie et des difficultés rencontrées lors de l'installation. Cependant, une fois cela fait nous avons déterminé que cela était la meilleure réponse à certains problèmes rencontrés avec le nuage pédagogique afin de pouvoir travailler dans les meilleures conditions (notamment pour la limite des 4h où la machine est allumée et le fait de nécessiter d'une connexion internet).

MariaDB a également nécessité un temps d'adaptation, notamment pour appréhender les UUID et certaines particularités par rapport au SQL classique. Cependant, nous n'avons rencontré aucun problème sur ce projet concernant la base de données, car MariaDB ne présente pas de différences majeures par rapport à la syntaxe SQL Server/MySQL.

Là où nous avons également dû nous adapter, c'est dans l'utilisation d'Adminer, alors que nous avions l'habitude d'utiliser PHPMysqlAdmin avec M. Legales pour visualiser nos bases de données. Adminer s'est révélé différent, mais à été finalement très facile à prendre en main.

Dans le choix de l'IDE, nous avons opté pour **Visual Studio Code**, éditeur reconnu pour sa légèreté, sa flexibilité et son vaste écosystème d'extensions (PHP Intellisense, Twig, Prettier...). Grâce à ces extensions, nous avons pu enrichir notre environnement de développement en ajoutant des fonctionnalités essentielles telles que l'autocomplétion avancée, le linting, la gestion de bases de données, le débogage intégré et même l'intégration avec Git.

2.3.2 Explications techniques de l'IHM

2.3.2.1 Explications globales

Comme expliqué ci-dessus nous avons utilisé une structure **MVC**, nous allons alors vous présenter comment nous avons organiser le code pour répondre à la problématique.

Tout d'abord il y a le dossier **Public** qui gère tout ce qui est envoyé ou reçu par l'utilisateur. Que cela soit les images, les polices d'écriture, le css ou le Java-script. Ce qui est stocké dans le dossier "assets". Mais dans le dossier public il y a aussi "**index.php**" qui permet de gérer l'affichage des pages. Il sert notamment de point d'entrée du site pour l'utilisateur. C'est-à-dire que ce dernier permet de rester sur : "FileSphere.com/index.php?page=..."

Cependant afin de faciliter la compréhension des liens, l'index ne s'affiche pas, en effet il y a juste "FileSphere.com?page=..."

Ce dossier est un de ceux qui stocke le moins et avec lequel l'on travaille le moins bien qu'il soit pilier du site. En effet nous n'avons pas eu à le modifier (mis à part à créer les dossiers dans lequel stocker les fichiers envoyés.)

Ensuite, les dossiers composant le dossier "src" (respectivement "controller", "entity", "model", "extension", "routing" et "service") permettent d'organiser le code selon l'architecture MVC.

```

/filesphere
├── /public
│   ├── index.php
│   ├── /assets
│   │   ├── /css
│   │   ├── /js
│   │   ├── /images
│   │   └── /upload
├── /views
│   ├── assignmentController
│   ├── assignments.html.twig
│   ├── authController
│   │   ├── login.html.twig
│   │   ├── register.html.twig
│   │   └── resetpassword.html.twig
│   ├── classController
│   │   ├── classes.html.twig
│   │   ├── classSectionAssignmentDetails.html.twig
│   │   ├── classSectionAssignment.html.twig
│   │   ├── classSectionAssignmentSubmissions.html.twig
│   │   ├── classSections.html.twig
│   │   ├── classStudents.html.twig
│   │   ├── updateClass.html.twig
│   │   ├── updateClassSection.html.twig
│   │   └── updateClassSectionAssignment.html.twig

```

```
|— components
|   |— changePasswordForm.html.twig
|— dashboardController
|   |— dashboard.html.twig
|   |— profile.html.twig
|— defaultController
|   |— 403.html.twig
|   |— 404.html.twig
|   |— 500.html.twig
|— email
|   |— accountCreation.html.twig
|   |— accountDeletion.html.twig
|   |— accountResetPassword.html.twig
|   |— accountResetPasswordByAdmin.html.twig
|   |— classJoin.html.twig
|— fileController
|   |— files.html.twig
|— userController
|   |— updateUser.html.twig
|   |— users.html.twig
|— blank.html.twig
|— dashboard.html.twig

— /src
|— /Routing
|   |— Router.php
|— /Entity
|   |— Assignment.php
|   |— AssignmentFile.php
|   |— AssignmentInstructionFile.php
|   |— File.php
|   |— SchoolClass.php
|   |— SchoolClassSection.php
|   |— SchoolClassStudent.php
|   |— Section.php
|   |— SectionAssignment.php
|   |— User.php
|— /Controller
|   |— AssignmentModel.php
|   |— FileModel.php
|   |— SchoolClassmModel.php
|   |— SectionModel.php
|   |— UserModel.php
|— /Model
|   |— AssignmentController.php
|   |— AuthController.php
|   |— DashboardController.php
|   |— FileController.php
|   |— SchoolClassController.php
|   |— UserController.php
|— /Service
|   |— DepedencyContainer.php
|   |— Mailer.php
|— /Extension
|   |— SessionExtension.php

— /vendor (via Composer)

— .env
— composer.json
```

L'architecture MVC est caractérisée ici par :

- **Controller** : Ce dossier contient les différents contrôleurs qui gèrent la logique métier de l'application. Chaque contrôleur correspond généralement à une entité ou une fonctionnalité précise. Ils reçoivent les requêtes de l'utilisateur, appellent les modèles nécessaires pour récupérer ou manipuler les données, puis passent ces informations aux vues pour l'affichage.
 - **Exemple : UserController** :
 - Contient des méthodes comme `getUserByEmail()`, `createUser()` et `updateUser()` pour permettre d'accéder à la gestion des utilisateurs et renvoyer la vue et le model à l'utilisateur pour avoir les données
 - Permet d'afficher tel ou tel vue html.twig en fonction des actions de l'utilisateur (si il est connecté, si il envoie un formulaire...)
- **Entity** : Ce dossier regroupe les classes représentant les entités de la base de données. Chaque entité définit la structure des objets manipulés dans l'application, en spécifiant leurs attributs et les relations entre eux. Ces classes facilitent la gestion des données en s'assurant qu'elles respectent le format attendu.
 - **Exemple : UserEntity** :
 - Permet d'instancier la classe User, ainsi que de la composer et pouvoir interagir avec les données.
 - Permet aussi d'avoir les Getter et les setter pour interagir avec les données des objets.
- **Model** : Les modèles sont chargés de l'interaction avec la base de données. Ils contiennent les requêtes SQL et les fonctions permettant de récupérer, insérer, modifier ou supprimer des données. Ce sont eux qui assurent la communication entre l'application et la base de données.
 - **Exemple : UserModel** :
 - Contient des méthodes comme `getUserByEmail()`, `createUser()` et `updateUser()` pour gérer les utilisateurs.
 - Permet d'exécuter les opérations CRUD sur la table des utilisateurs.
- **Routing** : Ce dossier est responsable de la gestion des routes. Il permet d'associer les URL aux contrôleurs correspondants afin de déterminer quelle action exécuter en fonction de la requête de l'utilisateur. Grâce à ce système, il est possible de structurer et d'optimiser la navigation sur le site.
- **Service** : Ce dossier contient des classes utilitaires qui apportent des fonctionnalités supplémentaires aux autres composants du projet. Par exemple, on y trouve l'envoi de mails et le conteneur de dépendances.

En structurant notre code de cette manière, nous avons pu assurer une meilleure maintenabilité et lisibilité du projet. L'utilisation du modèle MVC nous a permis de séparer clairement les responsabilités entre la logique métier, la gestion des données et l'affichage, ce qui facilite les modifications et l'évolution du site dans le futur.

2.3.2.2 Explications spécifiques au routeur

Le routeur, en tant que point central de gestion des entrées des utilisateurs, joue un rôle crucial dans la gestion d'informations. Lorsqu'un utilisateur effectue une requête, elle transite par le routeur, qui est responsable de rediriger cette requête vers le contrôleur approprié en fonction de l'action effectuée.

Voici le parcours de l'information :

1. **Réception de la requête** : L'utilisateur envoie une requête. Cette requête comprend par exemple des informations de formulaire.
2. **Analyse de la requête**: Le routeur va analyser l'URL et les paramètres associés pour déterminer quelle action doit être effectuée. Cela implique de faire correspondre l'URL à une route définie dans la configuration du site. Si une correspondance est trouvée, le routeur détermine le contrôleur et l'action qui seront exécutés. Si elle n'est pas trouvée, le routeur renverra une erreur 404 si elle n'existe pas et 500 si elle est présente mais qu'il n'arrive pas à agir avec.
3. **Redirection vers le contrôleur** : Une fois que le routeur a identifié la bonne route, il redirige la requête vers le contrôleur associé. Ce contrôleur va ensuite traiter la logique, récupérer les données nécessaires (par exemple, depuis une base de données), et préparer la réponse. Par exemple renvoyer vers la page d'authentification et donc vers le contrôleur `authController` et voir quelles actions vont être effectuées par l'utilisateur.
4. **Retour au client** : Après que le contrôleur ait exécuté l'action nécessaire, le résultat (généralement sous forme de réponse HTML, ou autre) est envoyé au client via le routeur. Ce dernier peut, en fonction de la configuration, effectuer d'autres manipulations avant de transmettre la réponse à l'utilisateur. Par exemple, afficher à l'utilisateur que la connexion est réussie s'il envoie le formulaire d'inscription/de connexion, ou bien juste afficher la page demandée !

Dans le cadre d'un routeur utilisant Twig, une fois la réponse générée par le contrôleur grâce au modèle, les templates Twig sont souvent utilisés pour rendre les vues. Twig prend en charge le rendu des données dynamiques (comme les informations récupérées par le contrôleur).

Ce processus fait du routeur une composante clé de l'architecture, car il assure non seulement la navigation des requêtes, mais aussi une gestion centralisée de l'entrée des utilisateurs vers les différentes parties du système. Ainsi que la sécurisation des données avec les rôles.

2.3.2.3 Explications spécifiques au modèle

Le modèle (ou **Model** en architecture MVC) représente la gestion des données et de la logique. Il est chargé de récupérer, manipuler et organiser les données avant qu'elles ne soient envoyées à la vue ou utilisées par le contrôleur. Le modèle joue donc un rôle fondamental dans les données du système. Permettant notamment de communiquer avec une base de données pour garder les informations utilisateurs sur le serveur.

Voici un aperçu du parcours de l'information au niveau du modèle :

1. **Requête du contrôleur** : Une fois que la requête de l'utilisateur a été redirigée vers le contrôleur par le routeur, ce dernier interagit souvent avec le modèle pour récupérer les données nécessaires. Le contrôleur appelle les méthodes du modèle pour interagir avec la base de données ou d'autres sources de données externes (fichiers, etc.). Ce sont notamment grâce à des requêtes SQL que l'on interagit avec une base de données, l'on intègre ces dernières dans le modèle.
2. **Interaction avec la base de données** : Le modèle est responsable de la logique d'accès aux données.
3. **Manipulation des données** : Une fois les données récupérées, le modèle peut effectuer des calculs, appliquer des filtres ou des transformations avant de les renvoyer au contrôleur. Cela permet de centraliser la logique et de garantir que le contrôleur reste principalement responsable de la gestion des informations et de la navigation, sans être encombré par des opérations de gestion de données.
4. **Retour des données au contrôleur** : Après traitement, le modèle transmet les données (ou objets de données) au contrôleur. Celles-ci peuvent ensuite être passées à la vue pour être rendues via Twig.

2.3.2.4 Explications spécifiques aux entity

Les entités, dans le cadre de la programmation orientée objet sont les classes qui posent l'interfaces de celles ayant le même nom. Ainsi que leurs actions. Elles sont souvent mères des Models expliqué plus haut comme tout deux ont besoin d'interagir l'un avec l'autre pour créer les objets.

1. **Représentation des données** : Une entité étant une classe qui correspond possiblement à la base de données. Par exemple une entité **Utilisateur** a des propriétés "id", "name", "email", "role", "password", "isActive", "reset_token", "reset_token_expires_at", "created_at", "updated_at". Ces propriétés permettent de manipuler les données dans le code.

2. **Gestion de la persistance** : Une entité permet d'instancier les objets, ainsi lorsqu'un utilisateur se connecte, php va instancier la session avec les données User récupérées dans entity/user.php.

3. **Interaction avec le modèle** : Les entités sont utilisées dans le modèle pour encapsuler les données. Le modèle, dans ce cas, va interagir avec ces entités pour effectuer des opérations sur les données.

En résumé, les entités sont des éléments clés qui permettent de manipuler les données, elles assurent une séparation entre le stockage physique des données et la logique de l'application.

2.3.2.5 Explications spécifiques des contrôleurs

Les contrôleurs ont une importance majeure dans le fonctionnement du site et des structures MVC (Modèle vue contrôleur) en étant responsables des requêtes. Ils assurent la liaison entre l'interface utilisateur et le modèle. C'est lui qui gère en majorité les redirections des actions effectuées par un utilisateur tout en envoyant le résultat à la vue. Sans lui, rien ne s'affichera.

Le parcours de l'information dans un contrôleur se fait de la sorte:

1. Réception de la requête : Lorsqu'un utilisateur envoie une requête HTTP (donc une requête qui peut être un formulaire avec ses différentes méthodes, appuyer sur un bouton pour rediriger.

2. Envoie des données au traitement : le contrôleur interagit souvent avec les modèles pour récupérer ou manipuler les données nécessaires. Cela peut inclure des opérations sur la base de données via les entités et des traitements d'informations. Le contrôleur, par exemple, peut interroger le modèle pour obtenir la liste de tous les utilisateurs via `getAllUser`.

3. Redirection et affichage de la vue : Dès que le controller finit son traitement de données, l'on peut rediriger vers la réponse adéquate pour l'utilisateur. Générant ainsi le code HTML à afficher sur la page. Dans certains cas, le contrôleur peut également décider de rediriger l'utilisateur vers une autre page ou d'afficher un message de confirmation ou d'erreur.

4. Gestion des erreurs : le controller sert aussi à traiter les problèmes de gestion de données. Par exemple si il y a de mauvaises données rentrées, l'on va tout d'abord vérifier que toutes les données respectent bien ce que l'on souhaite rentrer (imaginons pour la création d'une classe il faut que ce soit avec un professeur et que le professeur existe), avec, ou non, des élèves qui existent, avec une image potentiellement, avec une couleur en hexadécimal et avec un nom. (Par exemple essayer de rentrer un pdf dans les images causera que le fichier ne sera pas accepté.)

5. Sécurité : Le controller permet aussi de faire une première vérification des informations et un nettoyage de ces dernières grâce aux méthodes `"filter_sanitize_..."` et les `"filter_validate_..."` qui nettoient ce qui ne devrait pas être dans ces données. De la sorte, quelque chose qui ressemble à une insertion SQL sera stoppé.

2.3.2.6 Explications spécifiques à la vue

Face immergée de l'iceberg, la vue est une des choses qui est le plus importante. Car sans elle l'on ne peut pas avoir les réponses du code suite à nos actions. Pour cette dernière nous utilisons Twig, permettant de mieux gérer la logique de présentation.

1. **Rôle de la vue** : la vue est chargée de recevoir les données préparées par le controller et de les rendre de manière lisible et structurée pour l'utilisateur final. Elle transforme les données de tableau ou d'objets (envoyées par le controller) en visuel sous HTML. Twig de son côté nous permet de mettre des conditions, notamment dans le cas d'affichage de toutes les classes ou de tout les utilisateurs, où il faut effectuer l'affichage de chaque, en HTML natif il faudrait que chacun ai sa ligne de code, là nous pouvons intégrer une boucle FOR afin de le faire pour chaque classes / user dans la liste.
2. **Passage des données** : Étant le vecteur premier d'interaction avec l'utilisateur, c'est grâce à lui que nous récupérons les informations que nous donne l'utilisateur au travers d'un formulaire. Envoyant ainsi au controller qui l'envoi vers le model et si tout se passe bien, dans la base de donnée si ce sont des données à envoyer dans la base de donnée, et enfin ça renvoi la réponse à l'entrée des données.
3. **Rendu dynamique** : Twig permet de rendre les pages dynamiques en utilisant des balises et des expressions dans les fichiers de template. Les données envoyées par le contrôleur peuvent être intégrées dans le template en utilisant des variables Twig, par exemple :

```
<input type="checkbox" name="students[]" value="{{ student.id }}"
onchange="document.getElementById('student-count').innerText =
document.querySelectorAll('input[name=\'students[]\']:checked').length;">
```

Ici, `{{ student.id }}` est une variable passée depuis le contrôleur, qui sera remplacée par la valeur de la propriété identifiant de l'objet l'étudiant (appartenant, dans le contexte de ces lignes, à une classe)

4. Structures de contrôle : Twig offre une série de structures de contrôle puissantes permettant de gérer les conditions et les boucles dans les templates. Par exemple, vous pouvez afficher un message différent selon que l'utilisateur soit connecté ou non, ou créer une boucle FOR. Vous pouvez en voir dans `classes.html.twig` ligne 66 à 71.

En résumé, la vue, réalisée avec Twig, est responsable de la présentation des données traitées par l'application, en fournissant un rendu dynamique et sécurisé. Elle permet de séparer clairement la logique de présentation du reste du système, garantissant ainsi un code plus propre, plus facile à maintenir et à tester.

Bien évidemment, tout les détails du code sont dans le code source.

3. Les points critiques

3.1 Points positifs et adéquation avec le sujet

Notre projet répond non seulement pleinement aux exigences du sujet en intégrant toutes les fonctionnalités requises, mais il va également au-delà en proposant des améliorations et des fonctionnalités supplémentaires.

Gestion des utilisateurs : création, administration et gestion des comptes utilisateurs.

Authentification sécurisée : système de connexion avec gestion des sessions.

Réinitialisation de mot de passe : procédure sécurisée avec envoi d'e-mail.

Envoi d'e-mails automatisé : notifications emails..

Administration des données pédagogiques : gestion des classes, sections, élèves et devoirs.

Gestion des fichiers : système d'upload et organisation efficace des fichiers.

Sécurisation du site : mise en place de rôles distincts (Administrateur, Enseignant, Élève) pour un accès restreint aux fonctionnalités.

Interface intuitive et responsive : design ergonomique facilitant la navigation sur tout type d'appareil.

Base de données évolutive : architecture flexible permettant l'ajout de nouvelles fonctionnalités à l'avenir.

3.2 Points négatifs

Tout projet comporte inévitablement des points perfectibles, et le nôtre ne fait pas exception. Nous avons identifié plusieurs aspects pouvant être optimisés :

Qualité et documentation du code : Le code manque parfois de clarté en raison de commentaires insuffisants.

Optimisation de la base de données : Avec des connaissances plus avancées, nous pourrions améliorer la structure de la base en intégrant davantage de contraintes et de mécanismes d'héritage, afin d'optimiser la cohérence et la performance.

Suppression hiérarchique : Actuellement, la suppression de certains éléments, comme les classes, n'entraîne pas automatiquement la suppression des entités qui leur sont rattachées (sections, devoirs, fichiers, etc.). Cela peut générer des données orphelines dans la base, ce qui complique la gestion et la cohérence des informations stockées. À l'avenir, il serait essentiel d'implémenter une suppression hiérarchique afin d'assurer une cohérence des données. Ainsi, lorsqu'une classe est supprimée, toutes les entités qui lui sont liées (sections, devoirs, fichiers associés) seraient automatiquement supprimées.

Sécurisation des fichiers : Actuellement, les fichiers ne sont pas suffisamment protégés, ce qui signifie que n'importe quel utilisateur peut potentiellement accéder aux fichiers d'un autre, car le dossier d'upload est situé dans le répertoire public. À l'avenir, la mise en place d'un **système de protection renforcé** serait essentielle.

Ces axes d'amélioration nous offrent des perspectives d'évolution intéressantes pour renforcer la robustesse et la sécurité de notre projet.

Ressources documentaires et références utilisés

Les ressources suivantes ont été essentielles pour approfondir nos connaissances, optimiser certains aspects du projet et garantir une implémentation fluide et efficace :

EPSI : Référence principale pour récupérer le **style graphique** et assurer une **cohérence visuelle** à partir des couleurs.

OpenClassrooms : Plateforme qui nous a permis **d'approfondir nos connaissances en PHP**, de découvrir **de bonnes pratiques** et d'améliorer notre **approche du développement**.

Grafikart (YouTube) : Chaîne proposant des **tutoriels avancés**, des **bonnes pratiques** et une vision plus poussée du **développement web moderne en PHP**.

LinkedIn Learning : Ressource précieuse pour **explorer de nouvelles technologies**, se former sur **des outils spécifiques** et renforcer nos **compétences en gestion de projet**.

Documentation officielle des frameworks et outils (PHP/Twig/Docker...) : Lecture indispensable qui nous a permis **d'exploiter pleinement les fonctionnalités** de nos outils et d'assurer une **implémentation optimale**.

Stack Overflow et forums de développeurs : Plateformes incontournables qui nous ont aidés à **trouver des solutions** et à **résoudre des problèmes rencontrés** au cours du projet.

ANNEXES

Dictionnaire de données

USER

Code Attribut	Description	Nature	Type	Règle d'intégrité
user.id	UUID de l'utilisateur	Élément aire	CHAR(36)	Identifiant (not null, unique)
user.role	Rôle de l'utilisateur	E	ENUM('admin', 'teacher', 'student')	not null
user.name	Nom de l'utilisateur	E	VARCHAR(50)	not null
user.email	Email de l'utilisateur	E	VARCHAR(255)	not null
user.password	Mot de passe hashé de l'utilisateur	E	VARCHAR(255)	not null, unique
user.is_active	Booléen pour savoir	E	BOOLEAN	not null, default = 0
user.reset_token	token de l'utilisateur permettant de réinitialiser le mot de passe	E	VARCHAR(255)	default = null
user.reset_token _expires_at	date à laquelle le token ne sera plus valide	E	DATETIME	default = null
user.updated_at	La dernière fois que l'utilisateur a modifié son compte	E	DATETIME	default = CURRENT_TIMESTAMP
user.created_at	La date à laquelle le compte de l'utilisateur a été créé	E	DATETIME	default = CURRENT_TIMESTAMP

FICHIERS

Code Attribut	Description	Nature	Type	Règle d'intégrité
file.id	l'identifiant de fichiers dans la base de donnée	E	CHAR(36)	Identifiant
file.token	le token est ce qui permet de renommer dans la machine le fichier	E	VARCHAR(255)	unique
file.name	sauvergarde de l'ancien nom du fichier	E	VARCHAR(255)	not null
file.extension	L'extension du fichier	E	VARCHAR(10)	
file.size	La taille du fichier en octet	E	INT	not null
file.updated_at	la dernière fois que le fichier a été supprimé	E	DATETIME	default = CURRENT_TIMESTAMP
file.created_at	La date d'upload du fichier (de création dans la base de donnée)	E	DATETIME	default = CURRENT_TIMESTAMP

CLASSES

Code Attribut	Description	Nature	Type	Règle d'intégrité
class.id	l'identifiant de la classe	E	CHAR(36)	ID
class.updated_at	la dernière fois que la classe a été modifié	E	DATETIME	default = CURRENT_TIMESTAMP
class.color	la couleur de la classe	E	VARCHAR(7)	not null
class.created_at	la date de création de la classe dans la base de donnée	E	DATETIME	default = CURRENT_TIMESTAMP
class.name	le nom utilisé pour la classe	E	VARCHAR(50)	not null

SECTION

Code Attribut	Description	Nature	Type	Règle d'intégrité
section.id	Identifiant de la section	E	CHAR(36)	ID
section.name	Nom de la section	E	VARCHAR(255)	not null
section.updated_at	La dernière fois que la section a été modifié	E	DATETIME	default = CURRENT_TIMESTAMP
section.created_at	la date de création de la section dans la base de donnée	E	DATETIME	default = CURRENT_TIMESTAMP

DEVOIR

Code Attribut	Description	Nature	Type	Règle d'intégrité
Assignment.id	Identifiant du devoir	E	CHAR(36)	ID
assignment.name	Nom du devoir	E	VARCHAR(255)	not null
assignment.description	Description du devoir	E	TEXT	default = Null
assignment.start_period	Date de début du devoir	E	DATETIME	not null
assignment.end_period	Date de fin du devoir	E	DATETIME	not null
assignment.max_files	Nombre de fichiers maximums autorisé dans le devoir	E	INT	default = 1
assignment.allow_late_submission	Booléen d'autorisation à upload les fichiers après la fin du devoir	E	BOOLEAN	default = 0
assignment.updated_at	dernière fois que le devoir a été modifié	E	DATETIME	default = CURRENT_TIMESTAMP
assignment.created_at	date à laquelle le devoir a été créé	E	DATETIME	default = CURRENT_TIMESTAMP

Dépendances fonctionnelles

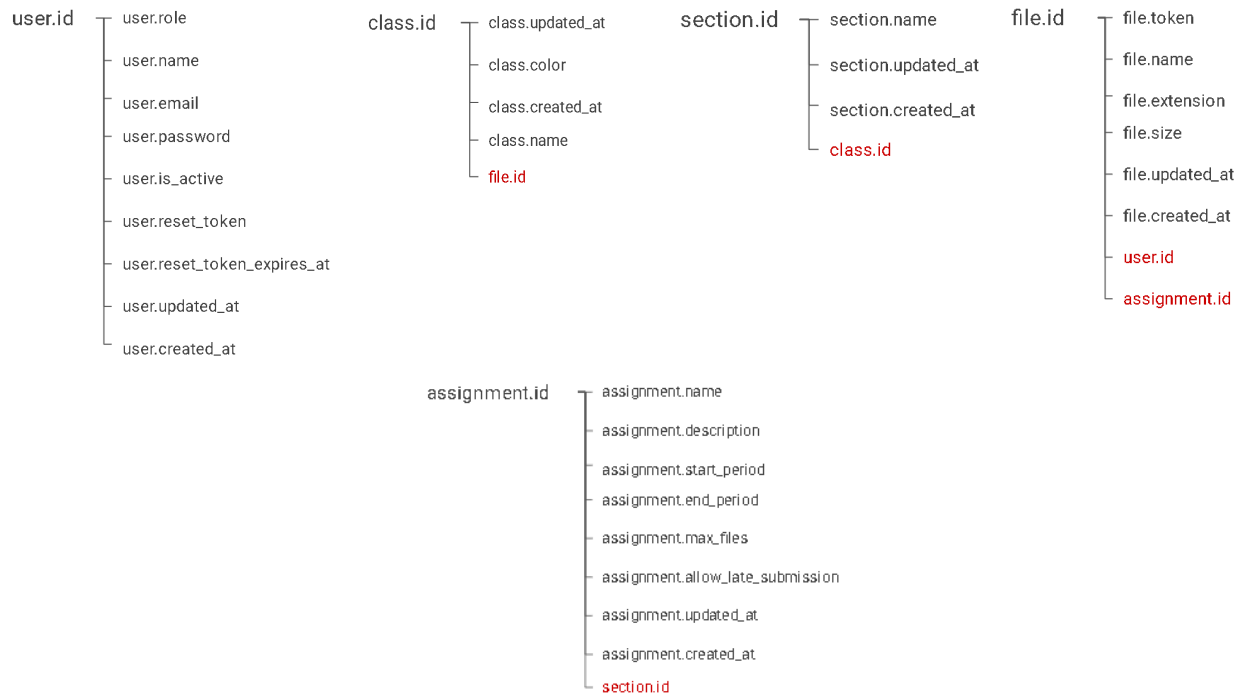


Schéma MCD

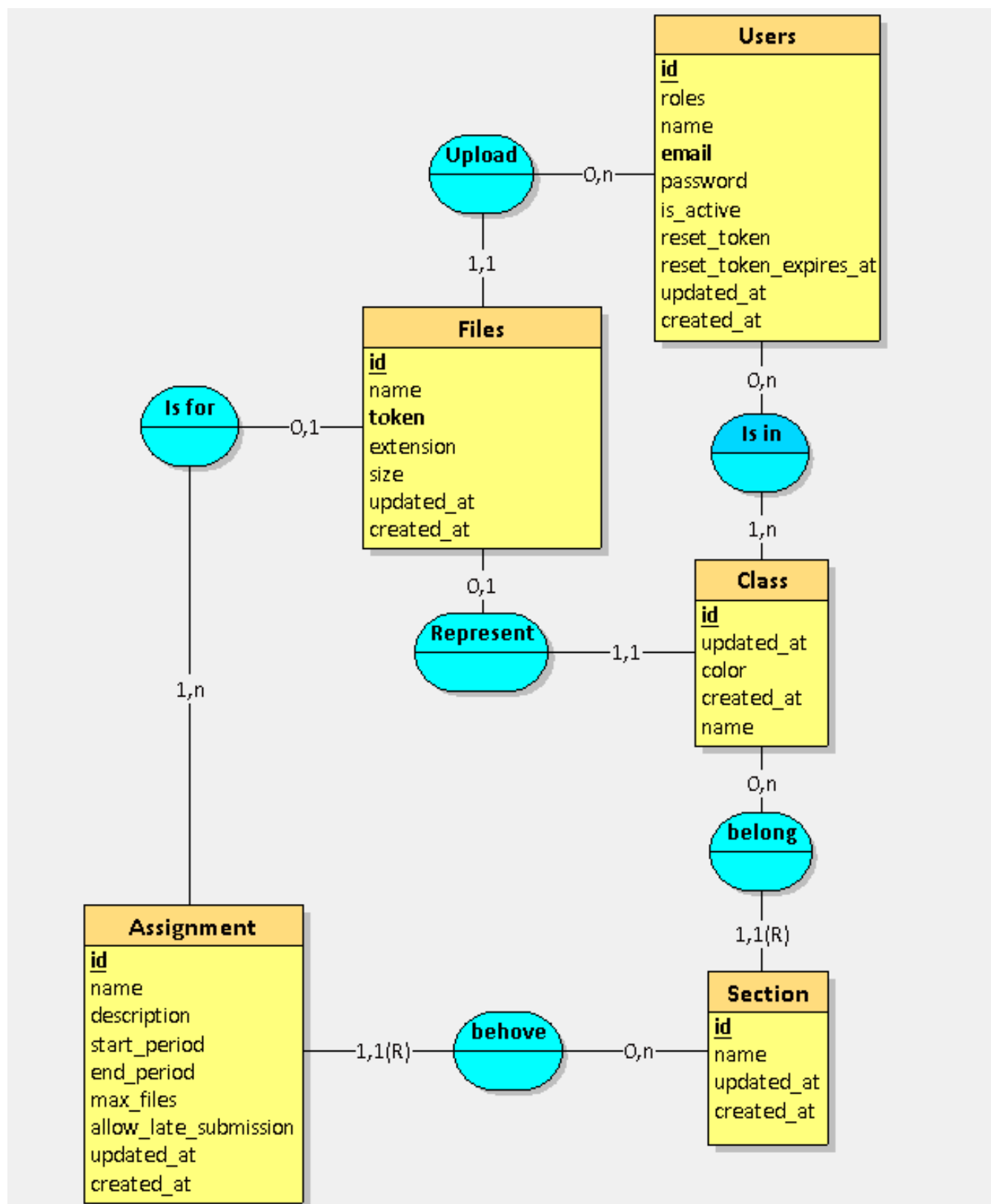


Schéma MLD

Users = (id COUNTER, roles VARCHAR(50), name VARCHAR(50), email VARCHAR(50), password VARCHAR(50), is_active VARCHAR(50), reset_token VARCHAR(50), reset_token_expires_at VARCHAR(50), updated_at VARCHAR(50), created_at VARCHAR(50));

Section = (#id_1, id VARCHAR(50), name VARCHAR(50), updated_at VARCHAR(50), created_at VARCHAR(50));

Files = (id VARCHAR(50), name VARCHAR(50), token VARCHAR(50), extension VARCHAR(50), size VARCHAR(50), updated_at VARCHAR(50), created_at VARCHAR(50), #id_1, #(#id_4, id_3, id_2)*);

Assignment = (#id_2, id_1, id VARCHAR(50), name VARCHAR(50), description VARCHAR(50), start_period VARCHAR(50), end_period VARCHAR(50), max_files VARCHAR(50), allow_late_submission VARCHAR(50), updated_at VARCHAR(50), created_at VARCHAR(50));

Class = (id VARCHAR(50), updated_at VARCHAR(50), color VARCHAR(50), created_at VARCHAR(50), name VARCHAR(50), #id_1);

SQL

```
-- Suppression des tables si elles existent
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS AssignmentInstructionFile;
DROP TABLE IF EXISTS AssignmentFile;
DROP TABLE IF EXISTS SectionAssignment;
DROP TABLE IF EXISTS ClassSection;
DROP TABLE IF EXISTS ClassStudent;
DROP TABLE IF EXISTS Assignment;
DROP TABLE IF EXISTS Section;
DROP TABLE IF EXISTS Class;
DROP TABLE IF EXISTS User;
DROP TABLE IF EXISTS File;
SET FOREIGN_KEY_CHECKS = 1;

CREATE TABLE User (
    id CHAR(36) PRIMARY KEY NOT NULL DEFAULT (UUID()),
    role ENUM('admin', 'teacher', 'student') NOT NULL,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT 0,
    reset_token varchar(255) DEFAULT NULL,
    reset_token_expires_at DATETIME DEFAULT NULL,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE File (
    id CHAR(36) PRIMARY KEY NOT NULL DEFAULT (UUID()),
    token VARCHAR(255) UNIQUE,
    name VARCHAR(255) NOT NULL,
    extension VARCHAR(10),
    size INT NOT NULL,
    user_id CHAR(36) NOT NULL,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES User(id) ON DELETE CASCADE
);

CREATE TABLE Class (
    id CHAR(36) PRIMARY KEY NOT NULL DEFAULT (UUID()),
    teacher_id CHAR(36) NOT NULL,
    file_id CHAR(36) DEFAULT NULL,
    color VARCHAR(7) DEFAULT '#ffffff',
    name VARCHAR(50) NOT NULL,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (teacher_id) REFERENCES User(id) ON DELETE CASCADE,
    FOREIGN KEY (file_id) REFERENCES File(id) ON DELETE SET NULL
);

CREATE TABLE ClassStudent (
    user_id CHAR(36) NOT NULL,
    class_id CHAR(36) NOT NULL,
    PRIMARY KEY (user_id, class_id),
    FOREIGN KEY (user_id) REFERENCES User(id) ON DELETE CASCADE,
    FOREIGN KEY (class_id) REFERENCES Class(id) ON DELETE CASCADE
);
```

```

CREATE TABLE Section (
  id CHAR(36) PRIMARY KEY NOT NULL DEFAULT (UUID()),
  name VARCHAR(255) NOT NULL,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Assignment (
  id CHAR(36) PRIMARY KEY NOT NULL DEFAULT (UUID()),
  name VARCHAR(255) NOT NULL,
  description TEXT DEFAULT NULL,
  start_period DATETIME NOT NULL,
  end_period DATETIME NOT NULL,
  max_files INT DEFAULT 1,
  allow_late_submission BOOLEAN DEFAULT 0,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE ClassSection (
  section_id CHAR(36) NOT NULL,
  class_id CHAR(36) NOT NULL,
  PRIMARY KEY (section_id, class_id),
  FOREIGN KEY (section_id) REFERENCES Section(id) ON DELETE CASCADE,
  FOREIGN KEY (class_id) REFERENCES Class(id) ON DELETE CASCADE
);

CREATE TABLE SectionAssignment (
  section_id CHAR(36) NOT NULL,
  assignment_id CHAR(36) NOT NULL,
  PRIMARY KEY (section_id, assignment_id),
  FOREIGN KEY (section_id) REFERENCES Section(id) ON DELETE CASCADE,
  FOREIGN KEY (assignment_id) REFERENCES Assignment(id) ON DELETE CASCADE
);

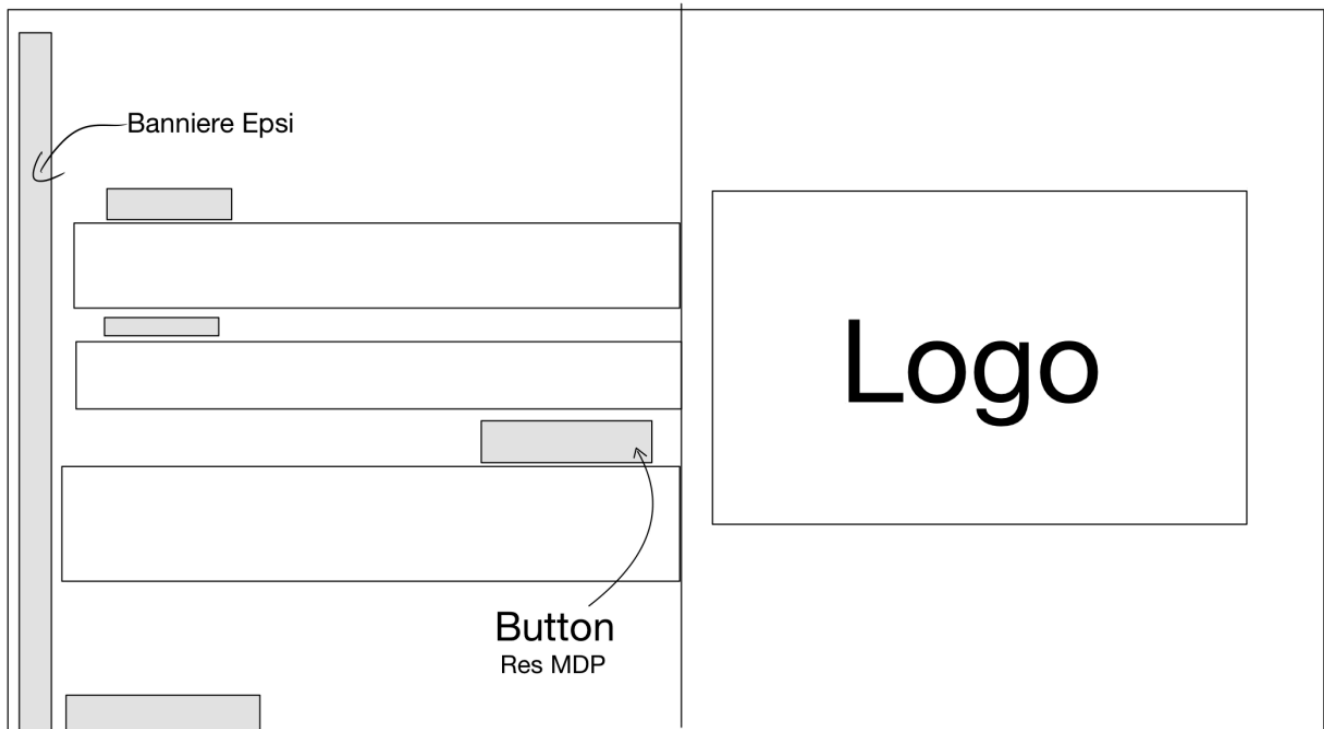
CREATE TABLE AssignmentFile (
  assignment_id CHAR(36) NOT NULL,
  file_id CHAR(36) NOT NULL,
  PRIMARY KEY (assignment_id, file_id),
  FOREIGN KEY (assignment_id) REFERENCES Assignment(id) ON DELETE CASCADE,
  FOREIGN KEY (file_id) REFERENCES File(id) ON DELETE CASCADE
);

CREATE TABLE AssignmentInstructionFile (
  assignment_id CHAR(36) NOT NULL,
  file_id CHAR(36) NOT NULL,
  PRIMARY KEY (assignment_id, file_id),
  FOREIGN KEY (assignment_id) REFERENCES Assignment(id) ON DELETE CASCADE,
  FOREIGN KEY (file_id) REFERENCES File(id) ON DELETE CASCADE
);

```

Wireframes

La page de login



Le tableau de bords

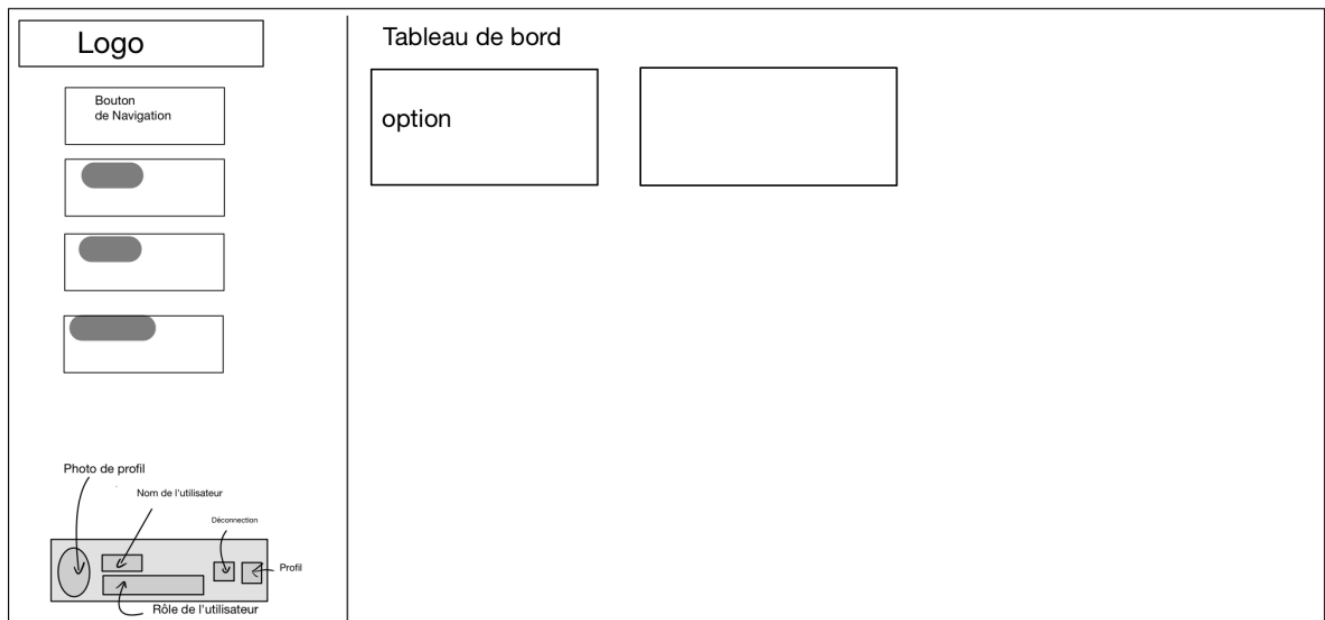


Tableau des classes

Logo

Bouton de Navigation

Photo de profil

Nom de l'utilisateur

Déconnection

Profil

Rôle de l'utilisateur

Classes

Photo de classe

Classe

Prof

000

Photo de classe

Classe

Prof

000

Photo de classe

Classe

Prof

000

Photo de classe

Classe

Prof

000

Photo de classe

Classe

Prof

000

Tableau des utilisateurs

Logo

Bouton de Navigation

Photo de profil

Nom de l'utilisateur

Déconnection

Profil

Rôle de l'utilisateur

User

Créer un Utilisateur

NOM	EMAIL	ROLE	ACTIONS
~	~	~	~

Tableau des fichiers

Logo

Bouton de Navigation

Photo de profil

Nom de l'utilisateur

Déconnection

Profil

Rôle de l'utilisateur

Fichier	Taille	Propriété	Date de mise en ligne	ACTIONS

Tableau des sections

Logo

Bouton de Navigation

Photo de profil

Nom de l'utilisateur

Déconnection

Profil

Rôle de l'utilisateur

Classes > classe > sections

Sections

Créer une section

Nom section000

000

000

000

000

000

000

000

Tableau des devoirs

Classes > classe > sections > section > Devoirs

créer une Devoir →

Nom Devoir	Date de rendu

Page Profil

Votre profil

Nom

Role

Email

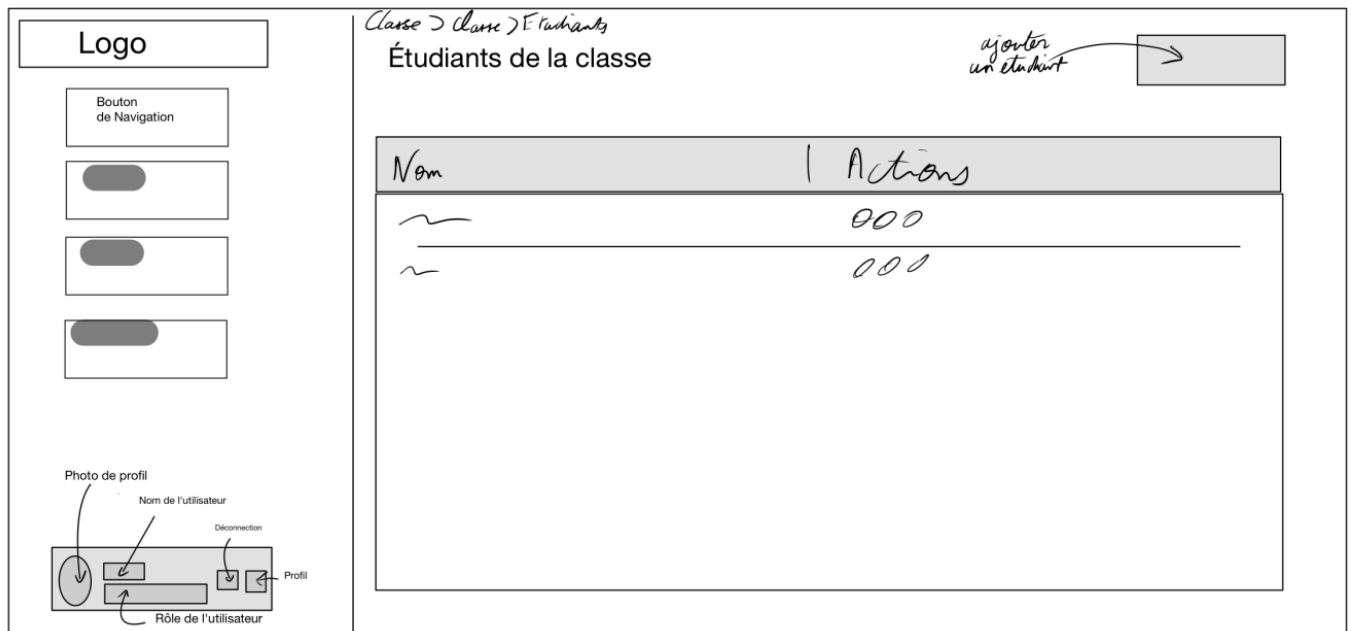
Changer MDP

Nouveau MDP

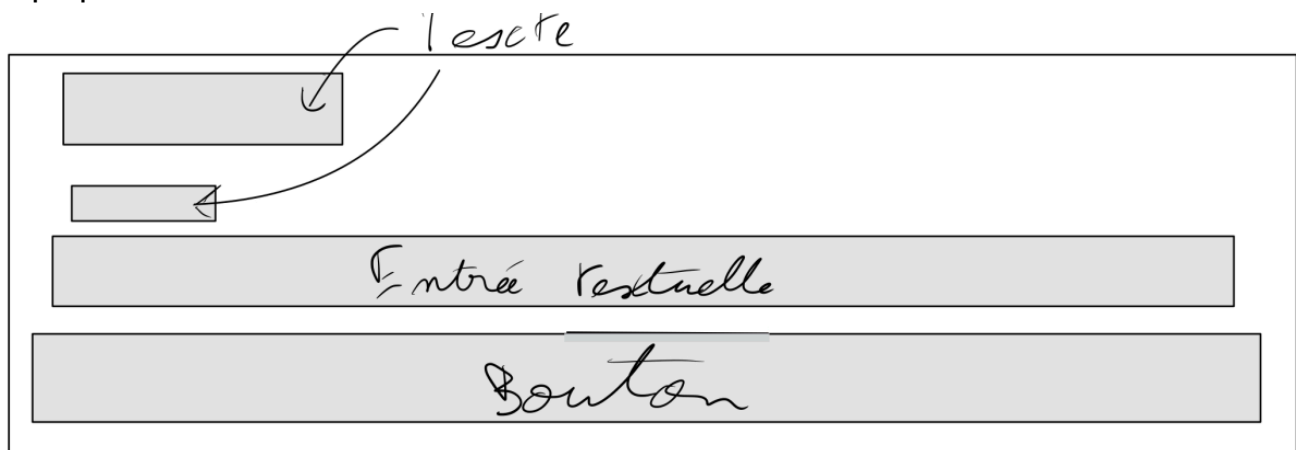
Confirmer Nouveau MDP

Modifier

Tableau des étudiants d'une classe



Pop-up de modification de donnée



Pop-Up sur une page

